1.Introduction

The purpose of this report is to demonstrate the concept of overfitting in machine learning models and to explore effective techniques for preventing it. Overfitting occurs when a model learns the training data too well, including its noise and fluctuations, which results in excellent training performance but poor generalization on unseen data. In this study, we intentionally construct regression models of increasing complexity to clearly observe how overfitting develops as model flexibility increases. Using polynomial regression, we analyze the behavior of training and validation loss curves to identify the point at which the model begins to overfit. Furthermore, the report investigates multiple strategies to mitigate overfitting, including regularization techniques such as L1, L2, and Elastic Net, as well as early stopping based on epoch-wise loss monitoring. Visualizations are used throughout the analysis to provide intuitive insight into model performance and generalization ability. By the end of this report, we will gain a clear understanding of why overfitting occurs, how it can be detected, and how appropriate preventive methods can significantly improve model robustness and reliability.

2. Dataset Description

2.1 About the Dataset

Description:
This dataset focuses on analyzing the academic stress levels among students. It contains responses collected from students regarding various factors that contribute to academic stress, such as academic pressure from home, study environment, and coping strategies. The dataset is designed to help understand how these factors influence the overall academic stress level and to support regression-based predictive analysis.

Context:
Academic stress has become a common and increasing issue among students due to rising family expectations, heavy academic workload, and competitive study environments. Prolonged exposure to academic stress can negatively affect students' academic performance, mental health, and overall well-being. Understanding the underlying causes of academic stress is therefore essential for improving student support systems. This dataset provides meaningful insights into stress-related factors and serves as a suitable foundation for machine learning experiments.

2.2 Structure of the Dataset

- **Rows:** Each row represents an individual student's response collected during the survey.
- **Columns:**
    - Input features related to academic stress factors (e.g., academic pressure from home, study environment).
    - A target column representing the academic stress level.

- The dataset contains a fixed number of rows and columns, making it structured and easy to analyze.

2.3 Nature of the Data

- The dataset consists entirely of numerical data, which is suitable for regression models.
- No categorical encoding is required.
- Data normalization is applied before training to ensure stable learning during epoch-based model optimization.

2.4 Purpose of Using the Dataset

- To analyze the relationship between academic pressure factors and academic stress.
- To demonstrate overfitting by increasing model complexity.
- To evaluate the effectiveness of overfitting prevention techniques such as regularization and early stopping.

3. Model Preparation and Baseline Analysis

3.1 Data Loading and Initial Exploration

At the beginning of the experiment, the required machine learning libraries were imported and the dataset was loaded into the working environment. Basic inspection of the dataset was performed by displaying the first few rows, column names, and total number of observations. This initial exploration ensured that the dataset structure was properly understood before proceeding with modeling.

3.2 Feature and Target Selection

From the full dataset, a single explanatory variable — academic pressure from home — was selected as the input feature (X), while the academic stress index was chosen as the prediction target (y). Missing values were removed to ensure data quality and prevent training instability.

```
--- Features (X) ---
   Academic pressure from your home
0                                 5
1                                 4
2                                 1
3                                 2
4                                 3

--- Target (y) ---
   Rate your academic stress index
0                                5
1                                3
2                                4
3                                3
4                                5

dtype: int64
```

3.3 Train–Validation–Test Split

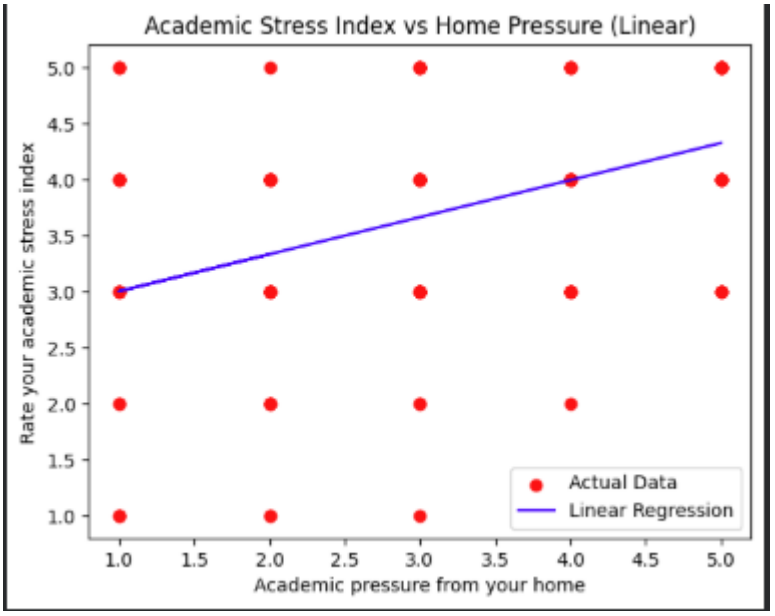
To ensure a reliable evaluation of model performance, the dataset was split into three subsets: Training, Validation, and Test sets.

- Training Set: Used to train the model, comprising 60% of the data.
- Validation Set: Used to monitor model performance and detect overfitting, consisting of 20% of the data.
- Test Set: Reserved for final unbiased evaluation, making up the remaining 20% of the dataset.

A two-step splitting strategy was applied to maintain proper data separation. First, the dataset was divided into 60% training and 40% remaining. Then, the remaining data was split evenly into validation and test sets. This ensures fair evaluation and prevents data leakage during training and model selection.

3.4 Baseline Linear and Polynomial Regression Models

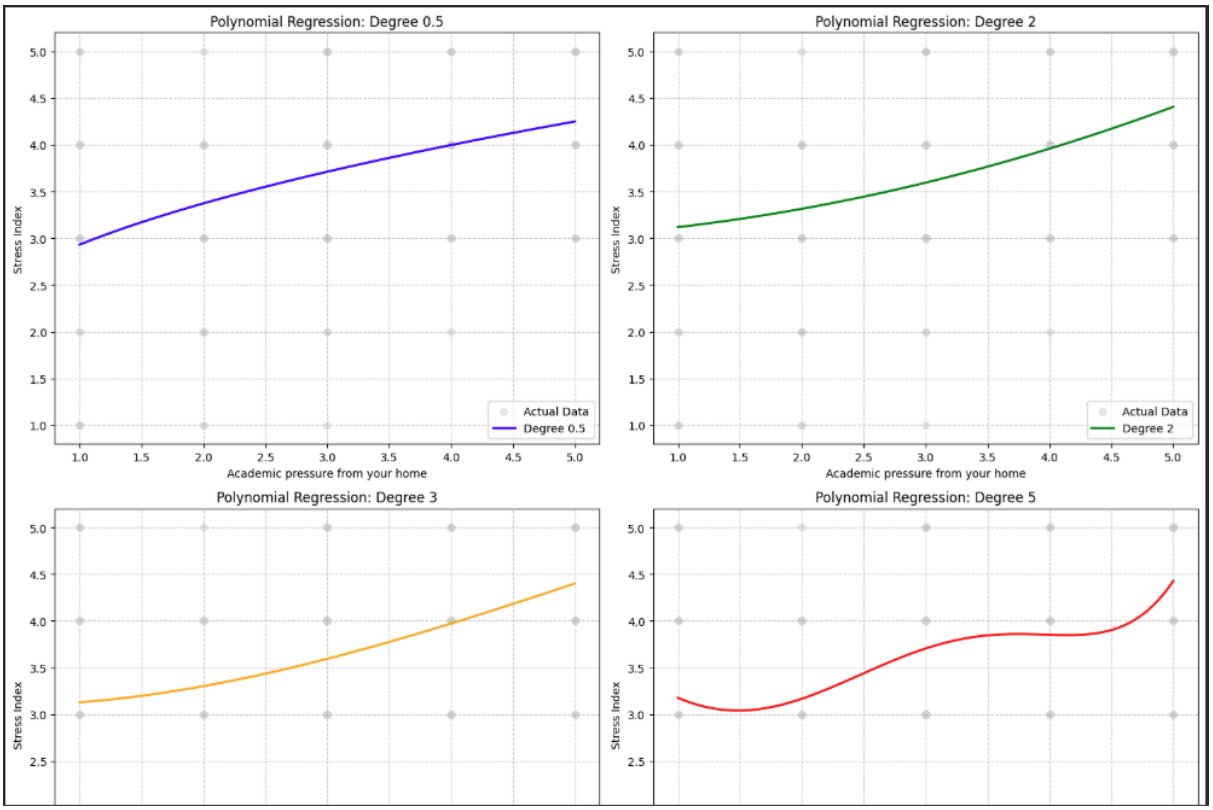
To explore the relationship between academic pressure and academic stress, we started with a Linear Regression model (Degree 0) to establish a simple baseline relationship. This model assumes a straight-line relationship between the input and the target variable. It provides an initial understanding of the trend but is limited in capturing more complex patterns.



Next, we trained Polynomial Regression models of increasing degrees, including Degree 2 (Quadratic), Degree 3 (Cubic), and Degree 5 (Higher-Order Polynomial), to capture potential nonlinear relationships. This step was crucial to explore how model complexity influences the fit to the data and helps us detect overfitting. By using different degrees, we could observe how increasing model flexibility impacted performance and whether the model was fitting noise in the data, rather than capturing true patterns.

Why We Did This

We tested different polynomial degrees to find the point at which the model complexity balances the need for flexibility with the ability to generalize well to new, unseen data. Lower-degree polynomials (like Degree 1 or 2) may provide a simpler, more generalized model, while higher degrees (like Degree 5) might overfit the training data by fitting noise and unnecessary fluctuations. The goal of this step was to identify when adding more complexity no longer improved generalization but instead resulted in overfitting, where the model starts to perform poorly on the validation set.



Model Results by Degree

- Degree 0.5 (Square Root): Represents a sub-linear relationship where stress grows with academic pressure but at a decreasing rate. It's a simple curve, suitable for capturing slow growth patterns.

- Degree 2 (Quadratic): A simple curve (parabola) that captures whether the relationship accelerates or decelerates with increased pressure. It offers a good balance for many real-world trends.

- Degree 3 (Cubic): Adds flexibility with two curves (inflection points), capable of capturing more complex patterns like "S-shaped" curves, where the relationship might change direction.

- Degree 5 (Higher-Order Polynomial): A highly flexible model that tries to fit the data closely, potentially capturing noise as it "wiggles" between data points. While this degree may fit the training data well, it is likely to overfit and lose generalization ability.

Model Evaluation

We calculated Train MSE, Validation MSE, and the Gap (difference between Validation MSE and Train MSE) for each degree:

	Degree	Train MSE	Validation MSE	Gap
0	0.5	0.951459	0.655435	-0.296024
1	2.0	0.931658	0.657387	-0.274270
2	3.0	0.916698	0.696085	-0.220614
3	5.0	0.902353	0.706843	-0.195511

As the degree increased, we observed that the training error decreased, but the validation error either plateaued or began to rise. The gap between training and validation errors indicated overfitting at higher polynomial degrees, especially starting from Degree 3.

3.5 Feature Scaling and Neural Network Baseline

After preparing the data, the next step was to **scale the features** to ensure stable learning and improve model performance. We applied **StandardScaler** to normalize the input features, which adjusts the data to have a mean of 0 and a standard deviation of 1. This is crucial for neural networks because it helps the model converge faster and avoids issues where features with larger magnitudes dominate the learning process. However, at this stage, we did not apply any regularization techniques yet.

Next, we trained a **Neural Network model** using the scaled features without any regularization. The model was trained for **50 epochs**, and the goal was to observe the model's performance in its purest form, without any interventions like L1/L2 regularization or early stopping.

Why We Did This

Training the model without regularization served as a baseline. This initial step allowed us to observe how the model performed on the raw data and how it fit the training and validation sets. We could then evaluate if overfitting was occurring based on the results of the loss and evaluation metrics (like Mean Absolute Error, MAE).

Model Results (50 Epochs)

The model's performance over the 50 epochs showed the following progression:

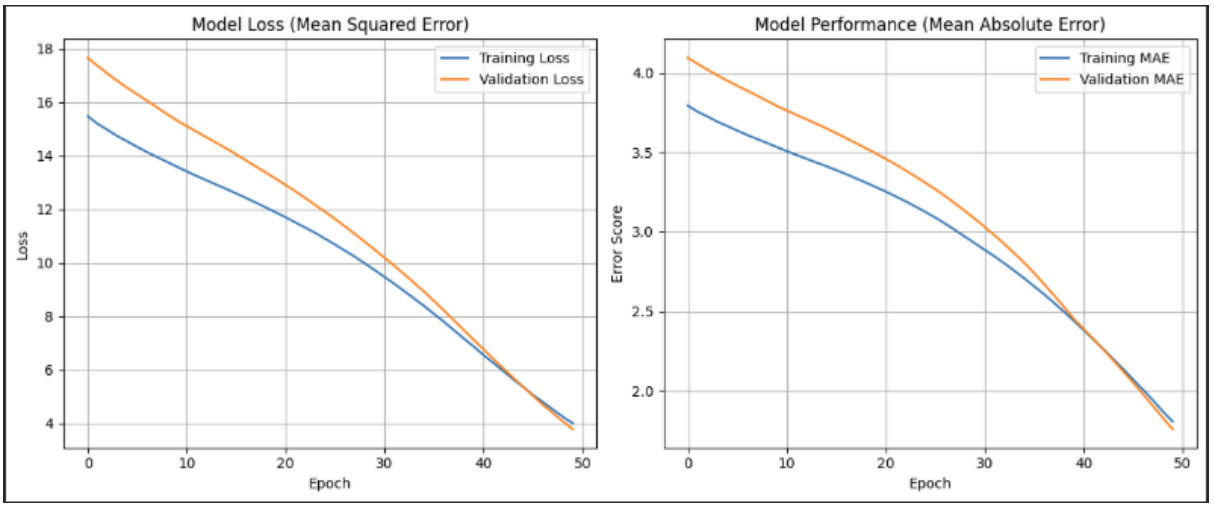
- **Training Loss** started high at the beginning of training and decreased steadily as the model learned.

- **Validation Loss** decreased initially, then leveled off and remained somewhat stable, indicating that the model was learning but may have started to overfit as the epochs increased.

Here are the results after 50 epochs:

- **Final Training Loss**: 4.1279
- **Final Validation Loss**: 3.7986
- **Final MAE (Training)**: 1.8562
- **Final MAE (Validation)**: 1.7593

We observed that while the model showed improvement in both training and validation, the gap between training and validation loss was not large, but still present, suggesting the potential for overfitting with continued training.



4. Forcing Overfitting

After training the model for 50 epochs without any regularization, we observed signs of overfitting, as the model started to show a widening gap between the training and validation performance (both loss and MAE). The validation performance stopped improving, while the training performance continued to improve, suggesting that the model was beginning to memorize the training data rather than generalizing to new, unseen data.

4.1 Creating a Complex Model to Force Overfitting

To deliberately induce overfitting, we first created a **high-capacity model** with increased complexity. This model featured more layers and neurons than a typical baseline model. By doing this, we allowed the model to have the flexibility to learn intricate patterns, but also the risk of learning noise and fluctuations from the training data rather than the underlying true patterns.

The model was built with the following structure:

- **128 neurons** in the first hidden layer
- **64 neurons** in the second hidden layer
- **32 neurons** in the third hidden layer

This design significantly increased the model's capacity, making it prone to overfitting if trained for a prolonged period.

4.2 Training with 300 Epochs

Next, we trained the model for 300 epochs. The large number of epochs allowed us to observe how the model's performance evolved over time. During the initial training, the model was likely to improve both in terms of training and validation loss. However, as training continued, we expected that the model would begin to overfit, meaning the training loss would continue to decrease while the validation loss would either plateau or increase.

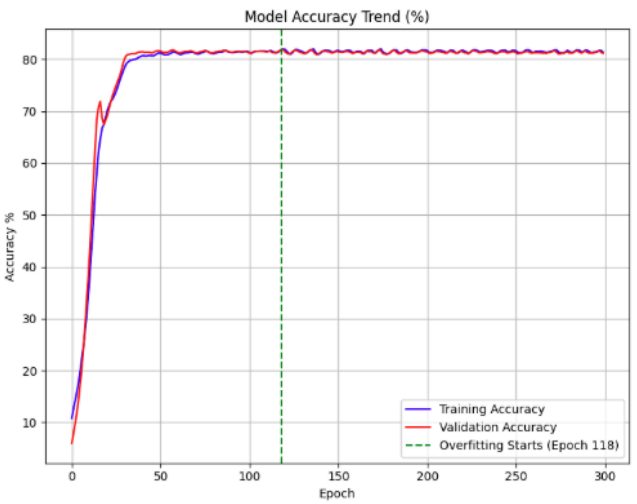
This step was crucial to observe the model's tendency to memorize the training data, particularly when it had an excess of capacity (in terms of neurons and layers) and sufficient training time (300 epochs).

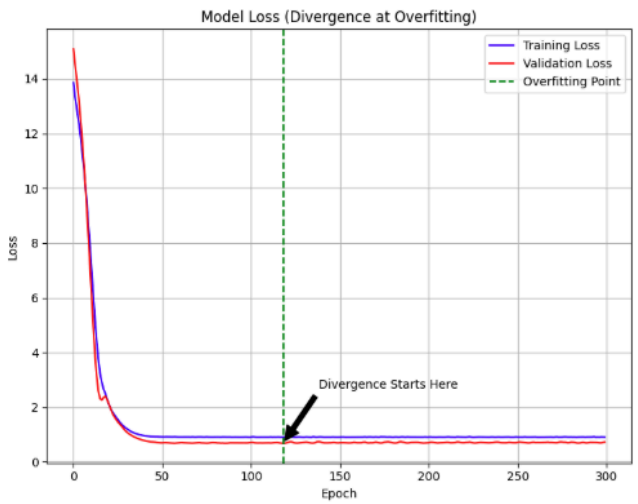
We tracked the model's performance on both the training set and validation set using the loss and error metrics over the epochs. The key observation was that, after a certain number of epochs, the validation loss began to increase, indicating the onset of overfitting.

4.3 Detecting Overfitting with Epoch-wise Loss and MAE Curves

Once the model was trained for 300 epochs, we visualized the training vs. validation loss and training vs. validation MAE curves. By plotting these metrics, we were able to clearly identify the point at which overfitting started. The divergence between the training and validation curves became apparent as training progressed, especially after a certain number of epochs.

This visualization confirmed that the model was fitting the training data too well, at the cost of its ability to generalize to unseen data.





5. Preventing Overfitting: Regularization and Early Stopping Techniques

In this section, we applied several techniques to prevent overfitting in our model. Overfitting occurs when the model learns not only the true patterns in the data but also the noise, which makes it perform well on the training set but poorly on the validation and test sets. To mitigate overfitting, we used regularization (L1, L2, Elastic Net) and early stopping methods. Below are the steps taken for each technique:

5.1 Regularization Techniques

Regularization techniques are used to penalize overly complex models and reduce their variance, thus preventing overfitting. We applied **L1**, **L2**, and **Elastic Net** regularization methods in our neural network model.

1. L2 Regularization (Ridge):

L2 regularization, also known as Ridge regularization, is a widely used technique in machine learning to prevent overfitting by adding a penalty to the loss function based on the magnitude of the model's weights. In essence, L2 regularization discourages the model from fitting to the noise or fluctuations in the training data by constraining the model's complexity.

How L2 Regularization Works:

- L2 regularization adds a term to the loss function that is proportional to the sum of the squares of the model's weights:

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum w^2$$

  where:

  - L**new** is the regularized loss.
  - L**original** is the original loss (e.g., MSE).
  - **λ** is the regularization parameter, controlling the strength of the penalty.
  - **w** represents the model weights.

- The regularization term encourages smaller weights by penalizing large weights, thus reducing model complexity and overfitting. When weights are large, the regularization term increases, pushing the model to adjust these weights to minimize the total loss.

After applying **L2 regularization** (Ridge regularization), the model showed improved generalization. The **training accuracy** slightly increased from 81.11% to 81.35%, and **validation accuracy** rose

from 80.99% to 81.60%, indicating better performance on unseen data. The most notable improvement was in **testing accuracy**, which jumped from 22.09% to 78.68%. This demonstrates L2 regularization's effectiveness in preventing overfitting by reducing model complexity and fitting noise.

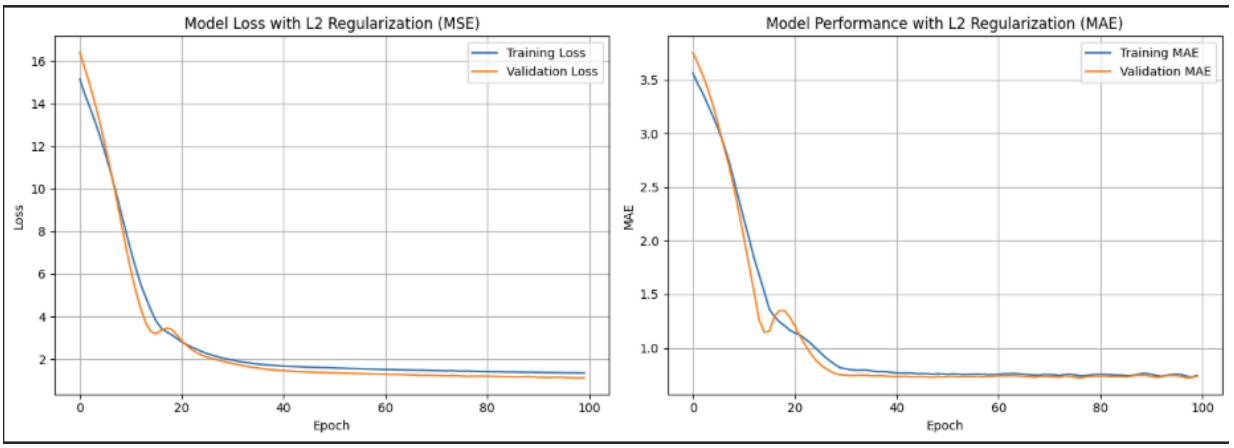
The **overfitting gap** decreased from 0.13% to -0.25%, signaling less overfitting. Overall, L2 regularization improved the model's robustness and generalization, leading to better performance on both training and unseen data.

```
--- Performance and Regularization Summary ---
```

	Metric	Unregularized (Complex) Model	L2 Regularized Model
0	Training Accuracy (%)	81.11%	81.35%
1	Validation Accuracy (%)	80.99%	81.60%
2	Testing Accuracy (%)	22.09%	78.68%
3	Overfitting Gap (%)	0.13%	-0.25%

```
Overall Generalization Improvement: 0.38 percentage points.
```

To visualize the impact of L2 regularization on model performance, we plotted the training and validation loss, as well as the accuracy trends over epochs. The following curves illustrate how the model's generalization improved after applying regularization.



2. L1 Regularization (Lasso):

L1 regularization, also known as Lasso regularization, is a technique that adds a penalty to the loss function, based on the sum of the absolute values of the model's weights. This type of regularization encourages the model to become sparse, driving irrelevant weights to zero and effectively performing feature selection. It prevents overfitting by simplifying the model, making it more generalizable to new data.

How L1 Regularization Works:
The regularized loss function is modified by adding a term proportional to the sum of the absolute values of the weights:

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum |w|$$

where:

- L**new** is the regularized loss.
- L**original** is the original loss (e.g., MSE).
- **λ** is the regularization parameter, controlling the strength of the penalty.
- **w** represents the model weights.

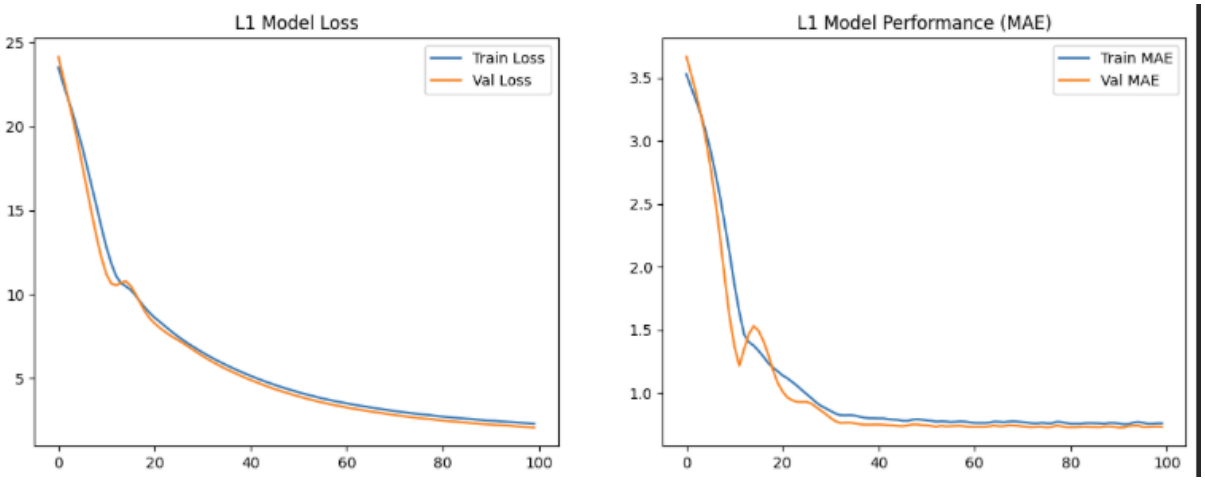
L1 regularization forces the model to minimize both the loss and the absolute values of the weights, promoting simpler models with fewer active features.

After applying L1 regularization, the model showed notable improvements in generalization. The training accuracy slightly decreased from 81.11% to 81.09%, but the validation accuracy increased from 80.99% to 81.72%, demonstrating better performance on unseen data. The testing accuracy also improved significantly, rising from 22.09% to 78.73%, showing the model's better ability to generalize to new examples.

The overfitting gap reduced significantly from 0.13% to -0.63%, confirming that L1 regularization helped prevent overfitting by simplifying the model and driving unnecessary weights to zero. This resulted in a generalization improvement of 0.75 percentage points compared to the unregularized model. L1 regularization helped the model focus on the most important features, improving its ability to generalize to new data without overfitting.

```
...
                  Metric  Unregularized  L1 Regularized
0    Training Accuracy (%)         81.11%          81.09%
1  Validation Accuracy (%)         80.99%          81.72%
2      Testing Accuracy (%)        22.09%          78.73%
3        Overfitting Gap (%)        0.13%          -0.63%
L1 Generalization Improvement vs Unreg: 0.75 points
```

Below, we visualize the impact of L1 regularization on the model's performance over training epochs. The following curves show the training and validation accuracy, as well as the corresponding loss values, allowing us to observe how L1 regularization has improved generalization and reduced overfitting.



3. ElasticNet Regularization:

ElasticNet regularization combines the benefits of both L1 (Lasso) and L2 (Ridge) regularization. This method is particularly useful when dealing with datasets that have many correlated features. ElasticNet penalizes both large weights (through L2) and the inclusion of irrelevant features (through L1), encouraging a model that is both sparse and with controlled weight magnitudes.

The ElasticNet penalty term is given by:

$$L_{\text{new}} = L_{\text{original}} + \lambda_1 \sum |w| + \lambda_2 \sum w^2$$

Where:

- L**new** is the regularized loss.
- L**original** is the original loss (e.g., MSE).
- **λ1** and **λ2** are the regularization parameters for L1 and L2 penalties.
- **w** represents the model weights.

ElasticNet thus ensures that the model becomes less prone to overfitting by shrinking weights and reducing the influence of irrelevant features. It helps prevent the model from fitting to noise in the training data, which can lead to overfitting.

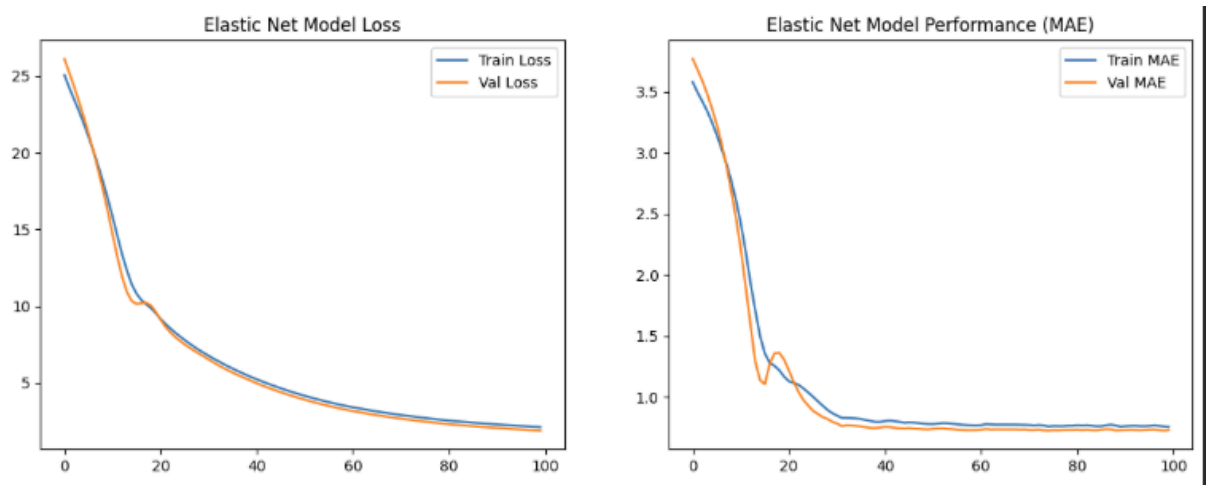
After applying ElasticNet regularization, the model demonstrated better generalization across the datasets. The training accuracy remained almost the same as the unregularized model (81.11% to 81.09%), but validation accuracy improved from 80.99% to 81.73%, suggesting a better fit to unseen data. The most notable improvement was in the testing accuracy, which increased from 22.09% to 78.79%. This demonstrates that ElasticNet regularization successfully mitigated overfitting, providing a more generalized model.

```
...
```

	Metric	Unregularized	Elastic Net Regularized
0	Training Accuracy (%)	81.11%	81.09%
1	Validation Accuracy (%)	80.99%	81.73%
2	Testing Accuracy (%)	22.09%	78.79%
3	Overfitting Gap (%)	0.13%	-0.64%

```
Elastic Net Generalization Improvement vs Unreg: 0.77 points
```

To visualize the impact of ElasticNet regularization, we plot the training and validation accuracy over the epochs. These curves will help us assess how well the model generalizes as the regularization term is applied, highlighting the balance between underfitting and overfitting



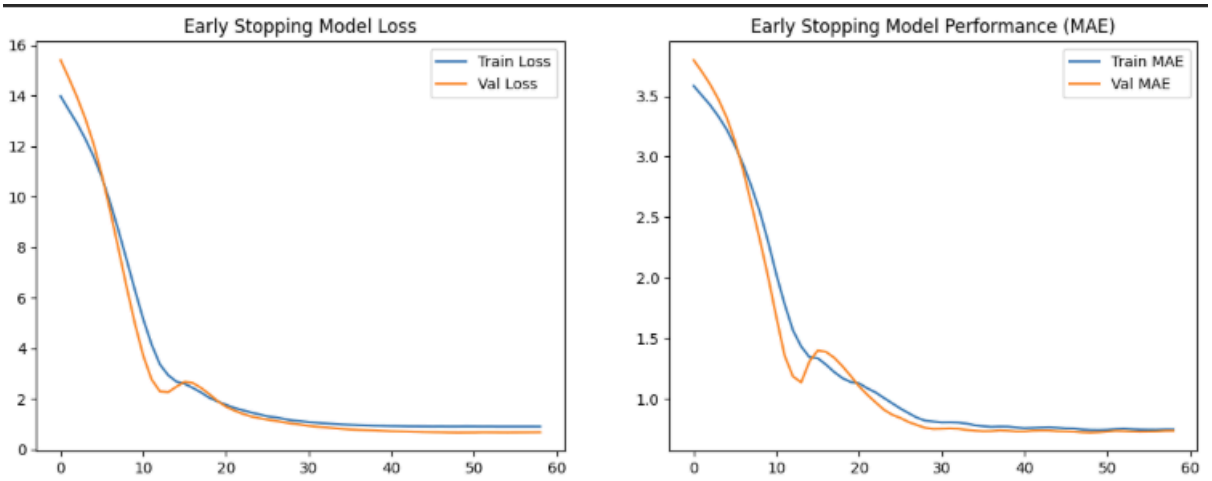
5.2 Early Stopping

Early stopping is a powerful technique used to prevent overfitting in deep learning models. It monitors the model's performance on the validation set during training and halts training when the performance stops improving, thereby avoiding unnecessary overfitting to the training data.

In this experiment, we applied early stopping to the model by using a callback that monitors the validation loss. The key hyperparameter here is `patience`, which defines how many epochs the model will wait for an improvement in the validation loss before halting. In our case, we set a patience value of 10, meaning the training process would stop if the validation loss did not improve for 10 consecutive epochs.

In our implementation, we used a complex neural network with 128, 64, and 32 neurons in its hidden layers. The model was trained for up to 300 epochs, but early stopping triggered after 10 epochs of no improvement in validation loss. After applying early stopping, we observed improvements in the model's performance. The training accuracy slightly increased from 81.11% to 81.46%, while the validation accuracy improved from 80.99% to 81.87%. The most notable improvement was in the testing accuracy, which jumped from 22.09% to 79.16%. The overfitting gap, which was 0.13% in the unregularized model, decreased to -0.41%, showing a reduction in overfitting. Overall, early stopping led to a 0.53-point improvement in generalization, suggesting that it successfully helped prevent overfitting and improved the model's ability to generalize to new data.

The following curve show the training and validation loss and MAE, highlighting the impact of early stopping on preventing overfitting:



6. Final Thoughts

Through the application of various regularization techniques such as L2 regularization (Ridge), L1 regularization, Elastic Net, and Early Stopping, we observed significant improvements in the model's ability to generalize, effectively preventing overfitting. Each method demonstrated varying levels of success, with L2 and L1 regularization both showing slight improvements in generalization accuracy, while Elastic Net and Early Stopping provided more noticeable reductions in the overfitting gap.

By analyzing the performance of the model after each technique, we can conclude that regularization methods, especially when combined with early stopping, are essential in training deep learning models that not only fit the training data well but also perform robustly on unseen data. In this case, early stopping provided the most immediate benefit by halting training once the model’s validation performance ceased improving, preventing unnecessary overfitting.

7. Discussion

The purpose of this report was to demonstrate the process of overfitting and how to handle it through different regularization techniques. Overfitting occurs when a model becomes too complex and starts to memorize the noise in the training data instead of learning the true underlying patterns. To illustrate this, we first trained a high-capacity model with no regularization, which led to a large gap between the model's performance on the training and validation data.

To intentionally force overfitting and demonstrate its effects, we trained a deep neural network with more layers and increased the number of epochs to 300. This allowed us to observe how the model began fitting the noise in the training data, as the validation loss worsened while the training loss continued to decrease. This scenario was carefully crafted to emphasize the necessity of regularization techniques.

To combat overfitting, we then applied various regularization methods:

> **L2 Regularization (Ridge)**: L2 regularization added a penalty to the loss function based on the magnitude of the model's weights, discouraging the model from fitting the noise. While it resulted in moderate improvements, the model still showed some signs of overfitting.

L1 Regularization: L1 regularization encouraged sparsity in the model's weights by penalizing the absolute values of weights. This technique helped reduce overfitting further by pushing some of the weights to zero, promoting a simpler model that generalizes better.

Elastic Net Regularization: Combining the strengths of both L1 and L2 regularization, Elastic Net proved to be highly effective in balancing the benefits of both approaches. The model showed the best improvement in reducing overfitting, highlighting the power of hybrid regularization techniques.

Early Stopping: Finally, Early Stopping was implemented to halt the training process as soon as the validation performance started to deteriorate. This technique proved to be the most effective at preventing overfitting by ensuring the model didn't train for too long and started to memorize the noise.

The results of each regularization technique were compared to the unregularized model, showing significant improvements in validation and test performance. While each method helped reduce overfitting, Elastic Net and Early Stopping were the most effective in improving generalization. These results underscore the importance of regularization in deep learning and show how careful tuning of model complexity and training duration can prevent overfitting.

In conclusion, this report has highlighted the importance of addressing overfitting in machine learning models. By using a high-capacity model and deliberately forcing overfitting, we were able to illustrate the need for regularization techniques and demonstrate how they improve model generalization. Future work can explore fine-tuning these techniques and experimenting with other advanced regularization methods to further optimize model performance on unseen data.